

Perguntas Prováveis · Apresentação P2

EasyGis — Banco de respostas preparadas

Engenharia de Software · 2026.1

Maio · 2026

Como usar este documento

As perguntas estão agrupadas em **8 blocos temáticos**. Cada pergunta tem:

- **Q:** a pergunta literal que o professor pode fazer.
- **A:** resposta preparada, curta e direta.
- **Quem responde:** o integrante com domínio do assunto.

Estratégia geral:

- Se a pergunta cair no seu slide, responda você. Se for sobre área de outro, passe a bola: “O X tem mais contexto sobre isso.”
- **Nunca chute.** Se não souber, diga “Não documentamos isso ainda, mas está em [doc X]” ou “Posso voltar com a resposta após a sessão.”
- **Cite o documento** sempre que possível (00-ENTREGA-P2.pdf, 04-diagramas-arquitetura.md, etc.).

Bloco 1 — Arquitetura

Q1.1 — Por que dois processos separados e não monólito?

A: Cada lado fala a linguagem ideal: Python tem o ecossistema geoespacial (rasterio, NumPy, GDAL); JavaScript renderiza mapa e 3D em WebGL. Acoplar tudo em um único processo nos forçaria a escolher um lado e perder o outro. REST + JSON é o contrato mais simples e debugável entre os dois mundos.

Responde: Vinicius Henrique.

Q1.2 — Por que não tem banco de dados?

A: Decisão consciente, registrada em **DA-02**. O escopo do MVP é single-user em localhost, processando arquivos .SAFE e KMLs já no filesystem. Adicionar banco aumentaria complexidade sem benefício imediato. Está no roadmap (próximo passo 02: SQLite ou PostGIS) quando virar multi-usuário.

Responde: Vinicius Henrique.

Q1.3 — Por que imagem em base64 dentro do JSON, e não um endpoint que retorna binário?

A: Por simplicidade do contrato: uma resposta única (**IndexResult**) traz a imagem, as estatísticas e o histograma juntos. O frontend processa em um só lugar. O custo do base64 é 33% de overhead, aceitável para imagens de tamanho (100 KB). Se virasse gargalo, trocaríamos para endpoint binário separado.

Responde: João Leonardi.

Q1.4 — O sistema é escalável?

A: Hoje não — é single-user em localhost. Mas a separação frontend/backend permite escalar horizontalmente o backend FastAPI atrás de um load balancer quando precisar. O processador é sem estado, então escala trivialmente.

Responde: Vinicius Henrique.

Bloco 2 — Stack tecnológica

Q2.1 — Por que FastAPI e não Django ou Flask?

A: FastAPI dá validação automática via Pydantic e gera OpenAPI em `/docs` sem código extra. Django é overkill (ORM, admin, autenticação nativos) para o que é essencialmente um endpoint de cálculo. Flask exigiria plugins (`flask-pydantic`, `flasgger`) para o mesmo nível.

Responde: Luis Gustavo.

Q2.2 — Por que Leaflet e não Mapbox / Cesium?

A: Leaflet é maduro, leve e **não exige token pago**. Mapbox GL cobra após cota. Cesium é overkill para terreno 2D — usamos Three.js diretamente para o 3D, com mais controle.

Responde: Luis Gustavo.

Q2.3 — Por que Next.js e não React puro?

A: O App Router do Next.js permite ler KMLs server-side via API routes (`/api/fields`), sem expor o filesystem ao cliente. Em React puro precisaríamos de um servidor separado só pra isso.

Responde: Luis Gustavo.

Q2.4 — Por que uv e não pip + requirements.txt?

A: `uv` é cerca de $10 \times$ mais rápido em instalação, gera lock file determinístico (`uv.lock`) e simplifica setup com um único comando `uv sync`. Trade-off: dependência extra a instalar — mitigado pela documentação no README.

Responde: Luis Gustavo.

Bloco 3 — Backlog e priorização

Q3.1 — Como vocês priorizaram o que entrar e o que ficar de fora?

A: Aplicamos MoSCoW. Tudo que era **Must Have** (núcleo de processamento — RF-01 a 11) entrou primeiro. Os 5 **Won't Have** (autenticação, persistência em banco, exportação, multi-usuário, integração Copernicus) foram excluídos do MVP **conscientemente**: o objetivo era validar o núcleo geoespacial em um semestre, não construir um SaaS completo.

Responde: José Melquíades.

Q3.2 — Por que 30/35 e não 35/35?

A: Das 35 histórias: 28 entregues completas + 2 parciais (download manual de imagem) + 5 conscientemente fora (Won't Have). Os 5 fora não falharam — **foram planejados como fora desde o início**. 85,7% é a entrega real do que prometemos.

Responde: José Melquíades.

Q3.3 — Como vocês garantem que o backlog está rastreado para os testes?

A: Matriz RF × US em 03-especificacao-requisitos-SRS.md §6. Cada caso de teste em `tests/` cita o RF coberto no docstring. RF-04, 05, 06, 10, 11, 20 têm cobertura direta.

Responde: Lauan Matheus.

Bloco 4 — Testes

Q4.1 — A cobertura global de 24% não é baixa demais?

A: À primeira vista sim, mas é **honesto e explicado**. Esse número é puxado para baixo pelos endpoints HTTP, que exigem um produto Sentinel-2 `.SAFE` real (1 GB) para serem exercitados. Onde a lógica de cálculo realmente vive — no `sentinel_processor.py` — a cobertura é **55%**. Declaramos o gap como **DT-03** no documento de entrega.

Responde: Lauan Matheus.

Q4.2 — O que vocês fariam para subir a cobertura?

A: Três coisas: **(1)** mockar o rasterio nos testes de endpoint para não depender de `.SAFE`; **(2)** criar fixtures de produto Sentinel-2 mínimo (uma cena recortada de poucos KB); **(3)** CI com GitHub Actions rodando a suíte em cada PR.

Responde: Lauan Matheus.

Q4.3 — O que são as “fixtures sintéticas em NumPy”?

A: Em vez de carregar um .SAFE real, geramos arrays NumPy com o formato esperado (`shape=(H,W)`, `dtype=float32`) e valores conhecidos. Permite testar NDVI, EVI, máscaras e estatísticas em milissegundos, sem I/O de disco.

Responde: Lauan Matheus.

Q4.4 — Tem teste de UI / E2E automatizado?

A: Hoje não. O **Nível 3** (manual) cobre o fluxo end-to-end durante a demo. Adicionar Playwright ou Cypress está no backlog futuro — foi deixado fora porque o custo de manter testes de UI brittle não compensava no tempo do semestre.

Responde: Lauan Matheus.

Bloco 5 — Algoritmos e processamento

Q5.1 — Por que NDVI e não outro índice como padrão?

A: NDVI é o índice mais estabelecido e validado em sensoriamento remoto agrícola — comparável a literatura, software comercial e produtos oficiais. EVI e SAVI estão implementados também (RF-06), mas NDVI é o *default* por familiaridade do usuário.

Responde: João Leonardi.

Q5.2 — Como vocês tratam pixels de nuvem ou água?

A: Hoje a máscara binária do polígono recorta apenas o talhão. Pixels de nuvem dentro do polígono entram no cálculo — limitação consciente. A próxima evolução é usar a banda QA60 (*cloud mask*) do Sentinel-2 para excluir esses pixels.

Responde: João Leonardi.

Q5.3 — Por que a classificação K-Means e não Random Forest?

A: K-Means é **não-supervisionado** — funciona sem rótulos de treinamento, que não temos para Picos-PI. Random Forest exigiria um dataset rotulado por agrônomo, fora do escopo do MVP. Está no roadmap (próximo passo 03) quando tivermos dados rotulados.

Responde: João Leonardi.

Q5.4 — O cálculo de área em hectares está correto?

A: Sim. Usamos a fórmula do **laço** (*shoelace*) sobre as coordenadas reprojetadas em UTM — sistema métrico nativo. Sem reprojeção, o cálculo em graus daria área inflada. Verificável contra área no Google Earth.

Responde: João Leonardi.

Bloco 6 — Demo / produto

Q6.1 — Que tipo de imagem o sistema aceita?

A: Produtos Sentinel-2 Level-2A no formato **.SAFE** (estrutura padrão da ESA). O usuário hoje baixa do Copernicus Hub manualmente; integração direta com a API está no roadmap (próximo passo 01).

Responde: João V. Castello.

Q6.2 — Quanto tempo leva pra processar uma imagem?

A: 5 a 30 segundos dependendo do tamanho do talhão e do índice. NDVI é o mais rápido (duas bandas); EVI usa três e é ligeiramente mais lento. A suavização gaussiana custa cerca de 1 s extra em talhões grandes.

Responde: João V. Castello.

Q6.3 — Posso desenhar talhões ou só importar KML?

A: Os dois. A versão atual aceita KML do Google Earth e também tem ferramenta de desenho de polígono no mapa Leaflet. O desenho não persiste entre sessões (limitação 1 do slide 13) — está no roadmap.

Responde: João V. Castello.

Q6.4 — E se a imagem tiver erro? Quebra a interface?

A: Não. O backend faz validação Pydantic da entrada e retorna **422 / 404 / 500** estruturado. O frontend exibe um toast com a mensagem de erro. Cobertura desse caminho está no nível de integração (FastAPI TestClient).

Responde: João Leonardi.

Q6.5 — A interface está acessível?

A: Componentes shadcn/ui (Radix por baixo) garantem WAI-ARIA básico — foco visível, navegação por teclado, contraste mínimo. Não rodamos auditoria Lighthouse formal — é uma melhoria de roadmap.

Responde: João V. Castello.

Bloco 7 — Processo e equipe

Q7.1 — Como vocês dividiram o trabalho?

A: Por afinidade técnica e área de especialização, registrada no **slide 14**: backend (João Leonardi), frontend (João Castello), dados (José), testes (Lauan), pitch (Lucas), docs (Luis Gustavo), manual (Sammuel), arquitetura (Vinicius). Cada um teve ownership claro de sua área e código revisado por par em PR.

Responde: João Leonardi.

Q7.2 — Usaram alguma metodologia ágil?

A: Sprints curtos com ciclo semanal informal — reuniões de alinhamento e merge de PRs. Sem Scrum formal (sem PO/SM definidos), mas com backlog priorizado e iteração visível no `git log`.

Responde: João Leonardi.

Q7.3 — Quanto tempo a equipe gastou no projeto?

A: Aproximadamente um semestre letivo. As 28 histórias entregues estão distribuídas em 50 commits. Histórico completo em `git log`.

Responde: João Leonardi.

Q7.4 — Tem code review?

A: Sim. Todas as features foram mergeadas via PR no GitHub ([luisg0c/EasyGis](https://github.com/luisg0c/EasyGis)). Cada PR tinha pelo menos um aprovador além do autor. Sem CI bloqueando merge — foi limitação consciente (próximo passo 07).

Responde: João Leonardi.

Bloco 8 — Documentação e decisões de design

Q8.1 — O que é um ADR?

A: Architecture Decision Record. É um registro curto que captura **contexto, decisão e consequências** de uma escolha técnica importante. Documentamos 11 deles, de DA-01 (estrutura geral) a DA-11 (uv em vez de pip). Localizados em `04-diagramas-arquitetura.md` § 9 e `00-ENTREGA-P2.pdf` § 3.

Responde: Luis Gustavo.

Q8.2 — Qual decisão técnica mais difícil vocês tomaram?

A: A da **DA-09** — suavização gaussiana preservando NaN. Aplicar gaussiano direto sangrava valores para fora do polígono criando halo. A correção pela máscara filtrada é não-óbvia e é o que distingue um resultado correto de um plausível-mas-errado.

Responde: Luis Gustavo ou João Leonardi.

Q8.3 — Vocês consideraram alternativas para a stack atual?

A: Sim. Considerei Django + DRF (rejeitada por overkill); Mapbox (rejeitada por custo); Cesium para 3D (rejeitada por complexidade); pip puro (rejeitada por velocidade). Cada rejeição está documentada como ADR ou na seção de comparativos do `04-diagramas-arquitetura.md` § 8.

Responde: Luis Gustavo.

Q8.4 — Como vocês garantem que a documentação não fica desatualizada?

A: Documentos versionados no mesmo repo do código. Mudanças arquiteturais relevantes exigem PR que atualize o ADR correspondente. Não temos verificação automatizada de drift — é um próximo passo (*docs-as-code* com lint).

Responde: Luis Gustavo.

Perguntas armadilha — atenção redobrada

- “Vocês usaram ChatGPT / Claude / IA?”

A: Sim, declarado abertamente no README.md. IA foi usada para tarefas difíceis (processamento geoespacial, debugging de TypeScript, refatoração). Todo código foi revisado, testado e entendido pela equipe. Não somos uma equipe que copia e cola — usamos como pair-programmer.

- “Por que tão poucos testes (28)?”

A: O número não é a métrica certa — o que importa é **o que está coberto**. Os 28 testes cobrem todos os casos de borda do processador (NaN, divisão por zero, máscara de água, polígono inválido). Mais testes redundantes não agregariam.

- “O sistema está em produção?”

A: Não — é um MVP em localhost. Foi escolha consciente do escopo da disciplina. Hospedar exigiria autenticação e persistência, ambos no roadmap.

- “E se eu quiser usar com outra imagem que não Sentinel-2?”

A: Hoje só Sentinel-2 .SAFE. O processador é agnóstico de fornecedor — qualquer raster com bandas RGB+NIR funcionaria — mas a camada de I/O é específica do formato .SAFE. Generalizar é refatoração de meia tarde.

- “Qual a diferença entre vocês e QGIS / ArcGIS?”

A: EasyGis é **focado em um caso de uso** (agricultura de precisão por índices) com **zero curva de aprendizado** — abrir e clicar. QGIS é generalista e exige conhecimento de SIG. Não competimos com QGIS — somos uma porta de entrada para quem não vai aprender SIG.

- “O que vocês fariam diferente se comesçassem hoje?”

A: Três coisas: (1) integrar Copernicus API desde o dia 1 (download manual virou gargalo); (2) CI desde o dia 1; (3) persistir talhões em SQLite — barato e adiciona valor de verdade pro usuário.

Lembrete final: se uma pergunta cair fora do que está aqui, a melhor resposta é “Excelente pergunta — está documentada em [doc X], mas posso voltar com a resposta após a sessão.” Nunca chute.